

Dear Team Kilo,

We have thoroughly examined your documentation and designs across the Authentication, Game Logic (Checkers, Connect Four, Tic-Tac-Toe), Leaderboard/Matchmaking, GUI and Networking modules. Below is a concise summary of our findings, along with suggested improvements, diagram corrections, two feature requests, and constructive feedback. All points are based solely on the reviews you provided.

General Critique of the Design

Strengths:

- I. **Modular Structure:** The overall design shows commendable modularity. Different components (authentication, game logic, networking, leaderboard/matchmaking and GUI) are separated clearly, making the system more maintainable.
- II. **Security Measures:** Hashed passwords and session management were noted as strong points in the authentication module. This indicates awareness of basic security standards.
- III. **Logical Flow:** The use cases and class diagrams for multiple games (Checkers, Connect Four, Tic-Tac-Toe) demonstrate a well-organized flow of player actions and system responses.

Weaknesses:

- I. **Unrealistic Deadlines & Lack of Debugging Phases:** Several planning documents (e.g., for game logic, leaderboard, matchmaking, and networking) either lack explicit debugging/testing periods or set overly tight schedules, increasing project risks.
- II. **Missing or Incomplete Rule Enforcement:** Certain critical checks-like detecting wins or draws, enforcing legal moves, or promoting pieces to kings in Checkers-are absent or insufficiently described in the use case documents.
- III. **Edge Case & Error Handling:** Multiple reviews highlight insufficient documentation for illegal moves, invalid input, disconnection scenarios, or failure modes (e.g., connection drops, matchmaking errors).
- IV. **Lack of Clarity in Certain Actor Interactions:** Leaderboard and matchmaking diagrams occasionally fail to define what happens if match creation fails or if data updates (like player history) encounter errors.

Diagram Corrections/Changes

Game Logic UML & Use Case Diagrams (Checkers, Connect Four, Tic-Tac-Toe):

Add Win/Draw Detection Steps: Ensure your diagrams show when and how the system checks for winning or drawing conditions.

Incorporate Illegal Move Handling: Whether it is skipping a turn for an invalid move or providing feedback, the diagrams should outline the system's response to incorrect player actions.

Promotions & Special Rules: For Checkers, clearly integrate piece promotion logic in the class diagrams and state diagrams.

Leaderboard & Matchmaking Diagrams:

Clarify Actor Interactions: Show how the system handles errors such as unsuccessful match creation or scoring updates.

Player History Tracking: Include additional attributes or associations in the class structure to log a player's match history, rank changes, or disconnection events.

Networking Module Diagrams:

Failure Modes and Error Handling: Reflect in your use case and class diagrams what should occur if a socket fails, if the server crashes, or if a client disconnects abruptly.

Client-Server Role Clarity: Explicitly depict responsibilities for each side (who validates data, who confirms connection, etc.) to avoid overlaps or ambiguity.

Feature Requests

Reconnect After Disconnection

This feature allows players to reconnect to an ongoing game within a short time if they experience a network interruption. The game pauses, and the disconnected player has a limited window (e.g., 1-2 minutes) to resume without forfeiting.

Expected benefits include preventing unfair losses due to temporary disconnections, improving user satisfaction, and reducing frustration for both players.

Friend Request System with Privacy Controls

Instead of automatically adding users to each other's friend lists, implement a friend request flow where users can accept or decline requests. Provide profile settings to limit who can send friend requests.

This adds a deliberate and more secure way to manage social interactions on the platform, enhancing privacy and preventing unsolicited requests.

Constructive Feedback

Integrate Adequate Debugging & Testing Sprints: Across your planning documents, add explicit buffer times for testing each feature, especially once you begin integrating the GUI, networking, and game logic modules.

Improve Edge-Case Documentation: Expand your textual use cases and diagrams to detail what happens during invalid input, disconnections, or system errors. This will make your system more robust and reduce ambiguity for developers.

Assign Clear Roles & Responsibilities: When adjusting deadlines, specify which team member handles which part of the module or subsystem. This clarity helps track progress and avoids duplicating efforts.

Enhance Readability & Design Patterns: While your class structures appear logical, consider including more consistent naming conventions and design patterns (e.g., Model-View-Controller in the game modules, or a well-defined client-server pattern in networking). This ensures long-term maintainability.

Closing Remarks

We commend your team on the solid foundation you have laid. With more explicit handling of edge cases, realistic timelines for debugging, and refined diagrams, your project will be well-prepared for successful implementation. We trust that applying these recommendations will strengthen both the quality and stability of your final product.

Sincerely,
Reviewing Team